

Table of Contents

1. What is ICMP and Why Does It Exist?	3
The problem ICMP solves:	3
ICMP's relationship with IP:	3
2. ICMP Message Structure	3
3. What is Ping?	4
What ping does step by step:	4
Reading ping output:	4
4. What is TTL (Time To Live)?	5
Why TTL exists:	5
Default TTL values by OS:	5
5. What is Traceroute?	5
How traceroute works the TTL trick:	5
6. ICMP vs Ping vs Traceroute: Comparison Table	6
7. IP Address Setup	7
8. Task 1: ICMP Echo Request/Reply	7
REQUEST PACKET: full field explanation:	9
REPLY PACKET: full field explanation:	10
9. Task 2: TTL Behavior	11
What was observed:	11
What would change with real internet traffic:	11
Why TTL differences matter in security:	12
10. Task 3: Unreachable Destination	12
What happened and why:	13
Three different failure modes and what each means:	13
11. Task 4: Traceroute / TTL Exhaustion	14
12. Task 5: Ping Sweep Across Internal Network	16
The three behaviors mapped out:	17
Why ping sweep is a foundational recon technique:	17
13. Future Use Where This Applies to Detecting Suspicious Activity	17
1. Host Discovery / Reconnaissance Detection:	18
2. ICMP Tunneling Detection:	18

3. ICMP Flood (DoS) Detection:	18
4. TTL Anomaly Detection:	18
5. Traceroute-Based Network Mapping by Attackers:.....	18
6. Dead Host vs Filtered Host: Evasion Detection:	19

1. What is ICMP and Why Does It Exist?

ICMP stands for **Internet Control Message Protocol**. It is defined in RFC 792 and operates at Layer 3 the Network layer sitting alongside IP, not above it. ICMP is not a transport protocol like TCP or UDP. You cannot open a socket and "send data" using ICMP the way an application sends data over TCP. Instead, ICMP is a **diagnostic and error-reporting protocol** it exists to let network devices and hosts report problems back to the sender when something goes wrong with IP packet delivery, and to let administrators test reachability and measure latency.

The problem ICMP solves: IP itself is unreliable by design. It makes no guarantee that a packet will arrive, and it has no built-in mechanism to tell the sender when something went wrong. A router dropped the packet, the destination didn't exist, the TTL expired, the fragment couldn't be reassembled. Without ICMP, packets would silently disappear and the sender would have no way to know why. ICMP fills this gap by carrying error messages and diagnostic information back to the source.

ICMP's relationship with IP: Every ICMP message is encapsulated inside an IP packet. The IP header's Protocol field is set to 0x01 to indicate the payload is ICMP. Despite being carried inside IP, ICMP is considered part of the IP layer itself not above it. This is why ICMP can report errors about IP packets; it's the same layer talking to itself across the network.

2. ICMP Message Structure

Every ICMP message has the same basic header:

Field	Size	Description
Type	1 byte	What kind of ICMP message is (0, 3, 8, 11, etc.)
Code	1 byte	Sub-classification within that Type
Checksum	2 bytes	Error detection computed over the entire ICMP message
Rest of Header	4 bytes	Varies by Type (Identifier + Sequence for echo, unused for others)
Data/Payload	Variable	Depends on message type

The Type field is the most important thing. It tells you exactly what happened:

Type	Code	Meaning
------	------	---------

0	0	Echo Reply (ping answer)
3	0	Destination Network Unreachable
3	1	Destination Host Unreachable
3	3	Destination Port Unreachable
8	0	Echo Request (ping sent)
11	0	Time Exceeded TTL expired in transit (used by traceroute)
11	1	Time Exceeded fragment reassembly time exceeded

3. What is Ping?

ping is a command-line tool that uses ICMP Type 8 (Echo Request) and Type 0 (Echo Reply) messages to test whether a host is reachable and measure round-trip latency. It is one of the first tools any network engineer or security analyst reaches for.

What ping does step by step:

1. Your machine builds an ICMP Echo Request packet (Type 8, Code 0) with an Identifier (to match this ping process) and a Sequence Number (starts at 1, increments per packet)
2. This is encapsulated in an IP packet addressed to the target IP
3. This is encapsulated in an Ethernet frame addressed to the next-hop MAC
4. The target receives it, recognizes the ICMP Echo Request, and immediately builds an ICMP Echo Reply (Type 0, Code 0) with the same Identifier and Sequence Number
5. The reply travels back the same path
6. Your machine receives it, matches the Identifier and Sequence Number to the original request, calculates the round-trip time, and prints the result

Reading ping output:

64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=2.45 ms

- 64 bytes: size of the reply packet
- icmp_seq=1: sequence number, confirms which request this replies to
- ttl=64: TTL value in the reply packet at the moment it arrived (already decremented by hops)
- time=2.45 ms: round-trip time from request sent to reply received

What ping -c 4 means: -c 4 tells ping to send exactly 4 packets then stop. Without -c, ping runs forever until you Ctrl+C.

4. What is TTL (Time To Live)?

TTL is a field in the IP header, not the ICMP header. It is an integer counter set by the sender when the packet is created. Every router that forwards the packet decrements TTL by 1. When TTL reaches 0, the router drops the packet and sends an ICMP Type 11 (Time Exceeded) message back to the original sender.

Why TTL exists: To prevent packets from circulating forever in routing loops. Without TTL, a misconfigured network could loop packets endlessly, consuming bandwidth indefinitely. TTL guarantees every packet eventually dies.

Default TTL values by OS:

OS	Default Starting TTL
Linux	64
Windows	128
Cisco routers	255
macOS	64

This is why TTL is useful for **OS fingerprinting** if you receive a ping reply with TTL=64, the sender is probably Linux/Mac. TTL=128 suggests Windows. TTL=54 suggests Linux that went through 10 hops ($64 - 10 = 54$). Attackers and defenders both use this to infer what OS is running on a target without any deep scanning.

5. What is Traceroute?

Traceroute is a tool that maps the path packets take from your machine to a destination, identifying every router (hop) along the way and measuring latency to each one. It exploits TTL exhaustion deliberately.

How traceroute works the TTL trick:

1. Send probe packet with TTL=1. First router decrements it to 0, drops the packet, sends back ICMP Type 11 (Time Exceeded). You now know the IP of the first router and the latency to it.

2. Send probe packet with TTL=2. First router decrements to 1, forwards it. Second router decrements to 0, drops it, sends ICMP Type 11. You now know the second hop.
3. Repeat, incrementing TTL by 1 each time, until the destination itself replies with ICMP Type 0 (Echo Reply) instead of Type 11 — meaning the packet reached its target.

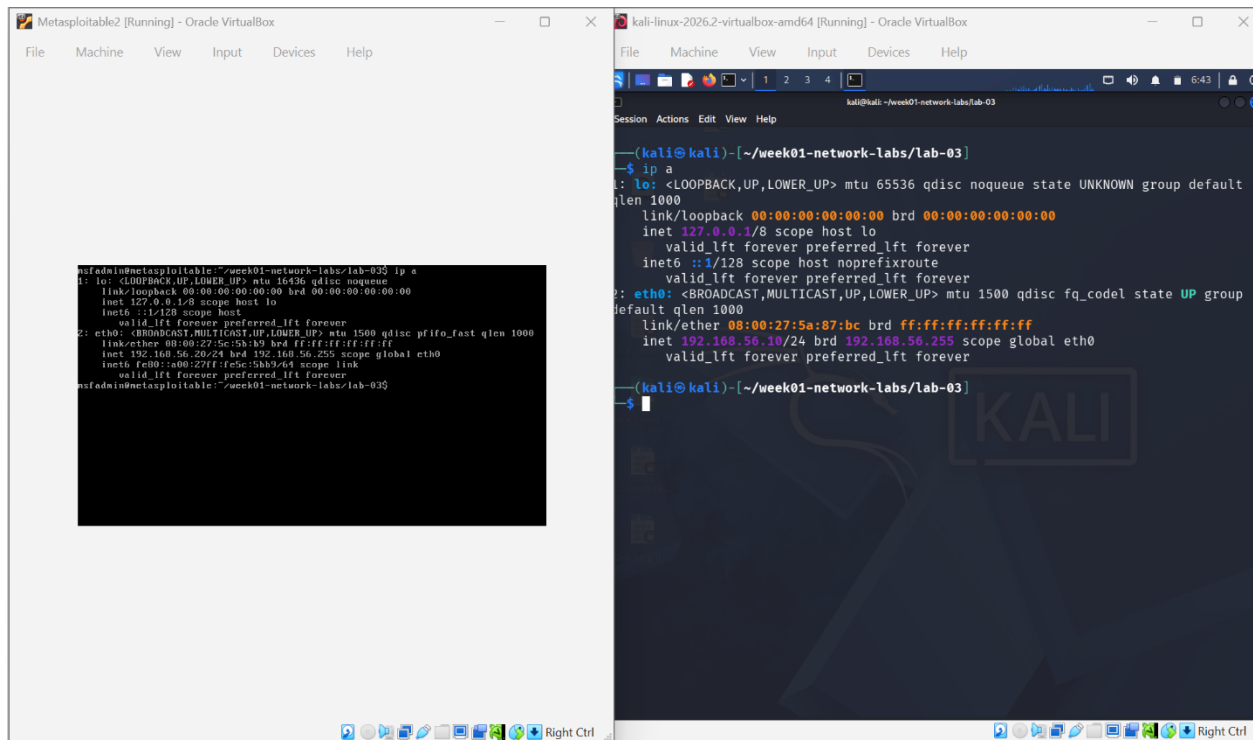
Result: A complete map of every router between you and the destination, with round-trip time to each.

Why * * * appears in traceroute output: Some routers are configured to silently drop packets with TTL=0 instead of sending an ICMP reply, or they rate-limit ICMP responses. When this happens, traceroute prints * * * for that hop meaning "a device exists here but it didn't respond." The path continues beyond it.

6.ICMP vs Ping vs Traceroute: Comparison Table

	ICMP	Ping	Traceroute
What it is	Protocol	Tool built on ICMP	Tool built on ICMP
Layer	3 (Network)	Uses Layer 3	Uses Layer 3
ICMP Types used	All types	Type 8 + Type 0	Type 8 (probes) + Type 11 (replies from routers) + Type 0 (final destination)
Primary purpose	Error reporting + diagnostics	Test reachability + measure latency	Map network path + identify each hop
Output	Raw packets	Reply/no-reply + RTT	Hop-by-hop IP list + latency per hop
Can detect	Network errors, unreachable hosts	Whether host is up, packet loss %	Network topology, routing issues, where packets die
Security relevance	ARP spoofing, ICMP tunneling, flood attacks	Host discovery in recon	Network topology mapping

7. IP Address Setup



Both VMs on Internal Network (intnet):

- Kali: 192.168.56.10
- Metasploitable2: 192.168.56.20

8. Task 1: ICMP Echo Request/Reply

Command used:

ping -c 4 192.168.56.20

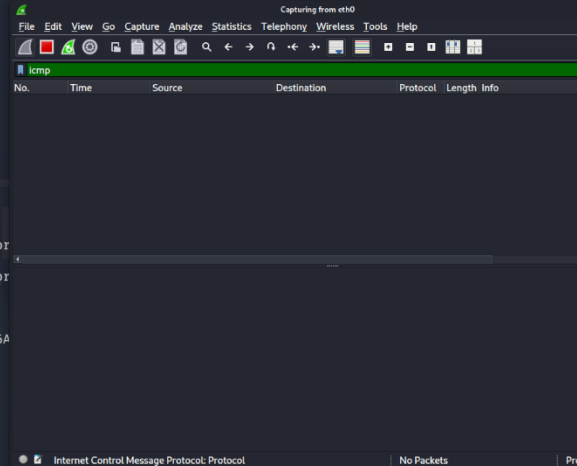
```
file machine view input devices help
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~/week01-network-labs/lab-03]
$ ping -c 4 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=12.0 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=4.32 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=2.88 ms
64 bytes from 192.168.56.20: icmp_seq=4 ttl=64 time=16.1 ms

— 192.168.56.20 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 2.883/8.823/16.056/5.435 ms
(kali@kali)-[~/week01-network-labs/lab-03]
$
```

Wireshark capture filter: icmp

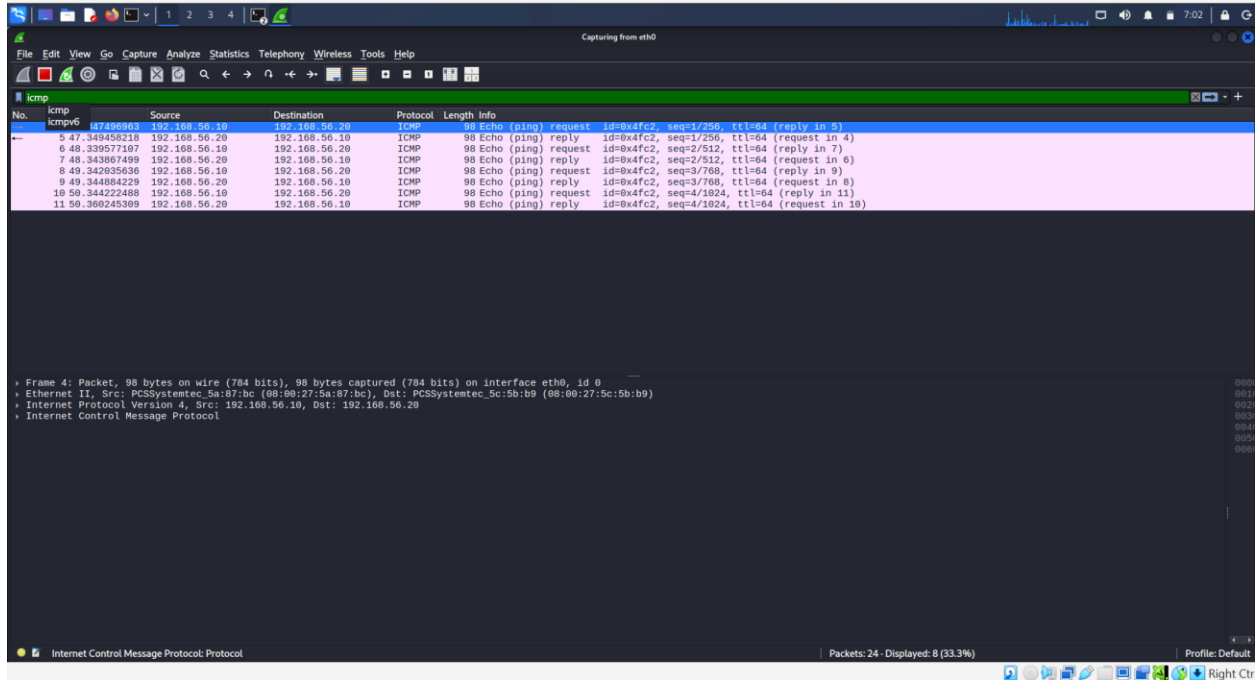
```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
file machine view input devices help
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~/week01-network-labs/lab-03]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global eth0
        valid_lft forever preferred_lft forever
(kali@kali)-[~/week01-network-labs/lab-03]
$ sudo wireshark
[sudo] password for kali:
** (wireshark:4430) 06:59:23.344930 [GUI WARNING] -- Failed to register with host por
er app ID: Connection already associated with an application ID*)
** (wireshark:4430) 06:59:24.341009 [GUI WARNING] -- Failed to register with host por
er app ID: Connection already associated with an application ID*)
** (wireshark:4430) 06:59:27.135367 [Capture MESSAGE] -- Capture Start ...
** (wireshark:4430) 06:59:27.217931 [Capture MESSAGE] -- Capture started
** (wireshark:4430) 06:59:27.217986 [Capture MESSAGE] -- File: "/tmp/wireshark_eth06A
[

```

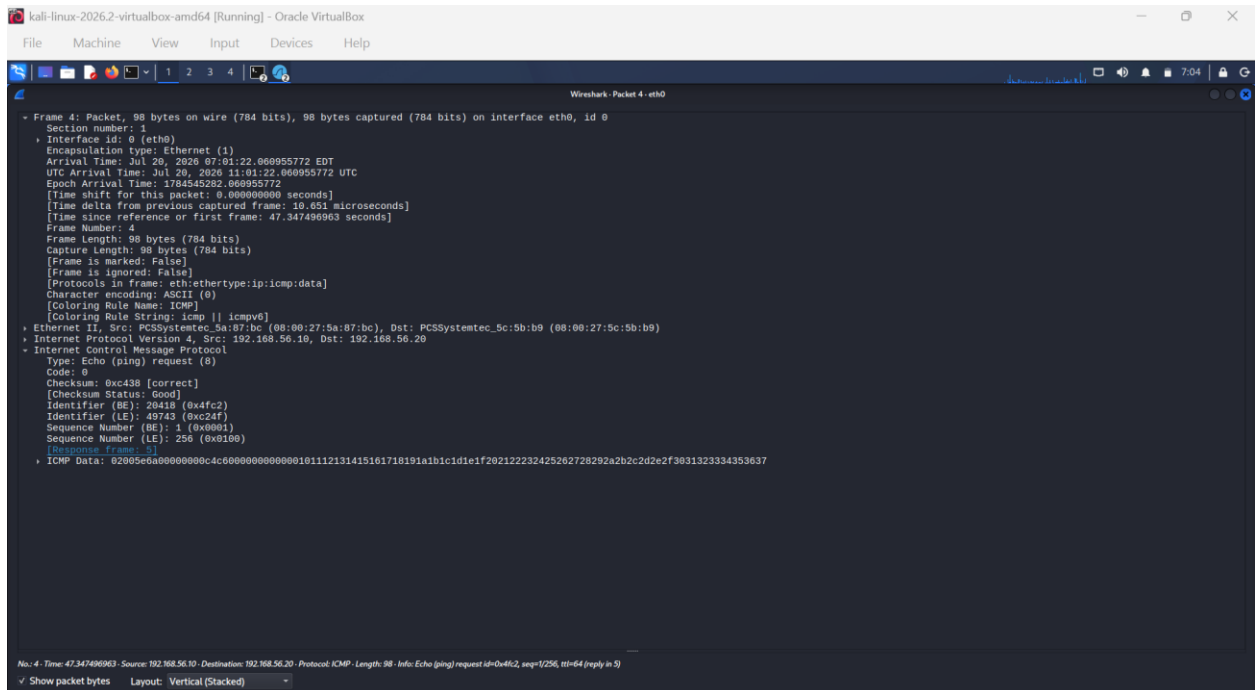


No.	Time	Source	Destination	Protocol	Length	Info
No Packets						

Why this command: `-c 4` sends exactly 4 ICMP Echo Requests. The capture filter `icmp` ensures only ICMP packets are stored no ARP noise, no background traffic. This gives a clean capture to inspect individual packet fields.



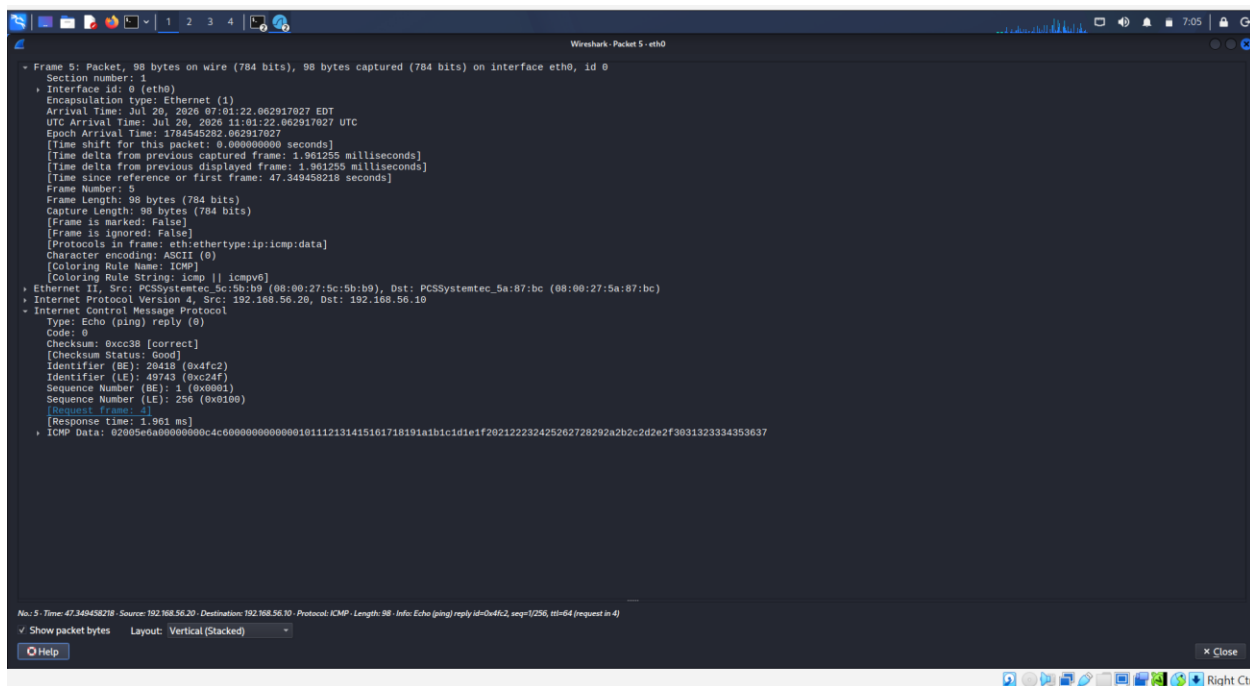
REQUEST PACKET: full field explanation:



Field	Value	Explanation
Type	8	Echo Request this is a ping being sent out
Code	0	No sub-classification for echo: always 0

Checksum	e.g. 0x4d4e	Computed over the entire ICMP message. Receiver recomputes and compares: mismatch = corrupted packet, discard
Identifier	e.g. 0x0001	A number assigned by the ping process to identify itself. If multiple pings are running simultaneously, each has a different Identifier so replies can be matched to the right process
Sequence Number	1, 2, 3, 4	Increments by 1 per packet. This is how ping tracks which reply corresponds to which request and detects out-of-order delivery or dropped packets
Data	timestamp + padding	ping includes a timestamp in the payload so it can calculate RTT when the reply arrives. The rest is padding to fill the packet to the requested size

REPLY PACKET: full field explanation:

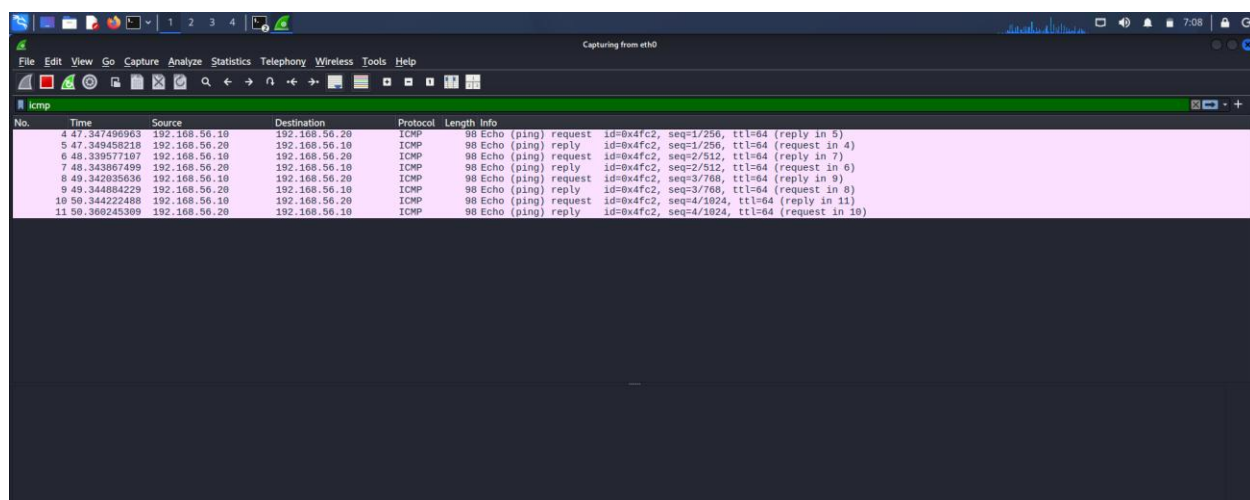


Field	Value	Explanation
Type	0	Echo Reply: this is the answer coming back
Code	0	Always 0 for echo reply
Checksum	recomputed	Metasploitable2 computed a new checksum for its reply

Identifier	same as request	Copied from the request so Kali can match this reply to its original ping process
Sequence Number	same as request	Copied exactly: 1 matches 1, 2 matches 2, etc. This is how ping confirms no packets were lost or reordered

What the output tells: 4 packets sent, 4 received, 0% packet loss means the path is clean and both machines are up and communicating. The RTT values (time= in ms) tell you how fast the network is between these two points.

9. Task 2: TTL Behavior



No.	Time	Source	Destination	Protocol	Length	Info
4	47.347495963	192.168.56.10	192.168.56.20	ICMP	98	Echo (ping) request id=0x4fc2, seq=1/256, ttl=64 (reply in 5)
5	47.348458218	192.168.56.20	192.168.56.10	ICMP	98	Echo (ping) reply id=0x4fc2, seq=1/256, ttl=64 (request in 4)
6	48.339577187	192.168.56.10	192.168.56.20	ICMP	98	Echo (ping) request id=0x4fc2, seq=2/512, ttl=64 (reply in 7)
7	48.343967459	192.168.56.20	192.168.56.10	ICMP	98	Echo (ping) reply id=0x4fc2, seq=2/512, ttl=64 (request in 6)
8	49.342935636	192.168.56.10	192.168.56.20	ICMP	98	Echo (ping) request id=0x4fc2, seq=3/768, ttl=64 (reply in 9)
9	49.344884229	192.168.56.20	192.168.56.10	ICMP	98	Echo (ping) reply id=0x4fc2, seq=3/768, ttl=64 (request in 8)
10	50.344222488	192.168.56.10	192.168.56.20	ICMP	98	Echo (ping) request id=0x4fc2, seq=4/1024, ttl=64 (reply in 11)
11	50.366245369	192.168.56.20	192.168.56.10	ICMP	98	Echo (ping) reply id=0x4fc2, seq=4/1024, ttl=64 (request in 10)

What was observed:

Looking at the IP header of packets in the same capture from Task 1:

- ICMP Request sent by Kali → TTL in IP header: **64** (Linux default)
- ICMP Reply sent by Metasploitable2 → TTL in IP header as seen by Kali: **64** (Metasploitable2 is also Linux-based, Ubuntu 8.04)

Why both show 64: Both machines are Linux-based so both start with TTL=64. Since they are on the same subnet (no routers between them Internal Network is a flat Layer 2 segment), zero hops are traversed. TTL is decremented only at routers, not at the final destination. So the TTL value Kali sees in the reply is the full 64 that Metasploitable2 started with — not decremented at all.

What would change with real internet traffic: If you pinged 8.8.8.8 (Google's DNS) and got TTL=119 back, you'd calculate: Google's Linux server started at 64... but $119 > 64$, so it started at 128 (Windows) and traversed 9 hops ($128 - 9 = 119$). This is passive OS fingerprinting from a single ping reply.

Why TTL differences matter in security:

- A sudden change in TTL to a previously known host could indicate the traffic is now coming from a different machine possible ARP spoofing or man-in-the-middle attack
- Unexpected TTL values can reveal the true number of hops to a target, exposing network topology
- Some IDS/IPS systems alert on anomalous TTL values as a detection signal

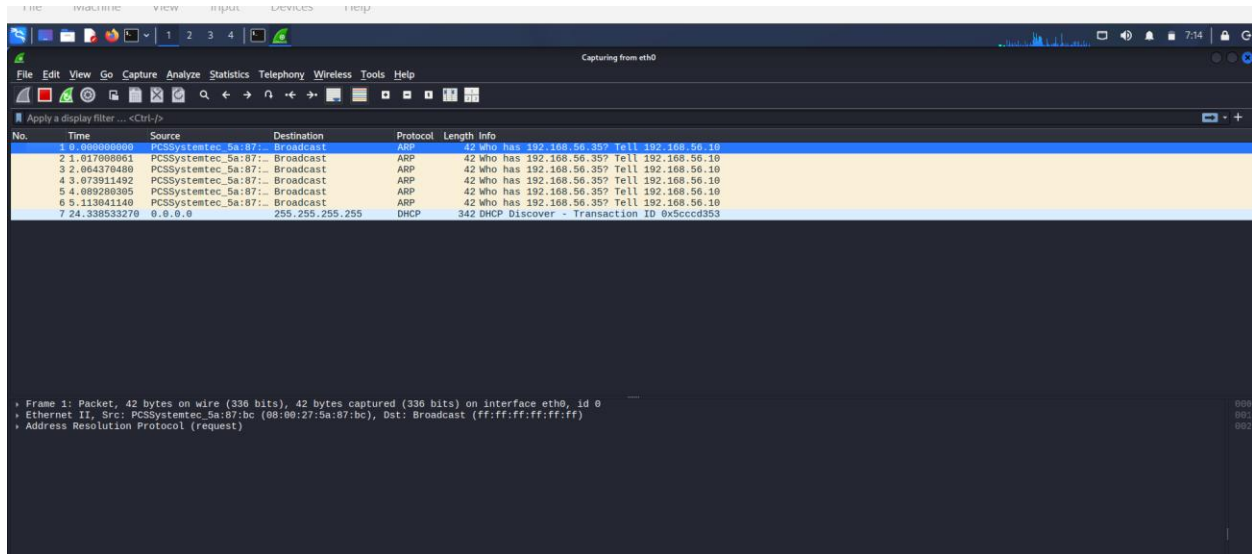
10. Task 3: Unreachable Destination

```
Session Actions Edit View Help
(kali@kali)-[~/week01-network-labs/lab-03]
$ sudo wireshark
** (wireshark:10646) 07:11:41.457523 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (wireshark:10646) 07:11:50.925235 [Capture MESSAGE] -- Capture Start ...
** (wireshark:10646) 07:11:51.071142 [Capture MESSAGE] -- Capture started
** (wireshark:10646) 07:11:51.071181 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0UU98R3.pcapng"
ping -c 192.168.56.35
^C

(kali@kali)-[~/week01-network-labs/lab-03]
$ ping -c 4 192.168.56.35
PING 192.168.56.35 (192.168.56.35) 56(84) bytes of data:
From 192.168.56.10 icmp_seq=1 Destination Host Unreachable
From 192.168.56.10 icmp_seq=2 Destination Host Unreachable
From 192.168.56.10 icmp_seq=3 Destination Host Unreachable
From 192.168.56.10 icmp_seq=4 Destination Host Unreachable

— 192.168.56.35 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3109ms
pipe 3

(kali@kali)-[~/week01-network-labs/lab-03]
$
```



The screenshot shows the Wireshark interface with a packet capture on the eth0 interface. The packet list shows several ARP requests from PCSSystemtec_5a:87:bc to the broadcast address ff:ff:ff:ff:ff:ff. The packet details pane shows the first packet, which is an ARP request. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
2	1.617088961	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
3	2.084370480	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
4	3.073911492	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
5	4.089280385	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
6	5.113041140	PCSSystemtec_5a:87:bc	Broadcast	ARP	42	Who has 192.168.56.35? Tell 192.168.56.10
7	24.338533278	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x5cccd353

Command used:

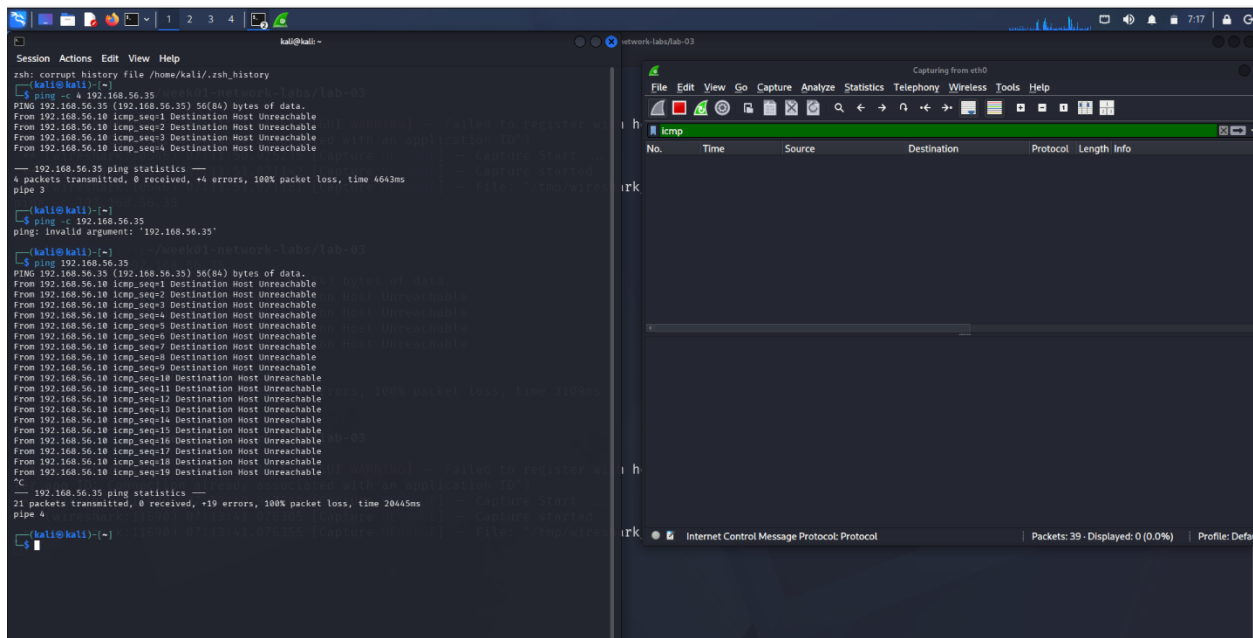
ping -c 4 192.168.56.35

(An IP within the subnet that no device is using)

What happened and why:

Kali's kernel checked its routing table. The address 192.168.56.35 is within the 192.168.56.0/24 subnet — same network as Kali. So, the kernel attempts to resolve the MAC address via ARP before sending the ICMP packet. It broadcasts "Who has 192.168.56.35?" repeatedly. Nobody answers because no device with that IP exists on the network. After ARP timeout, the kernel reports to ping: **"Destination Host Unreachable"** and this is the error ping prints.

In Wireshark you will see: ARP Requests going out (broadcast), no ARP Replies coming back, and no ICMP traffic at all because the ICMP Echo Request was never sent. Kali couldn't build the Ethernet frame without a destination MAC, so ping never even got to send the ICMP packet.

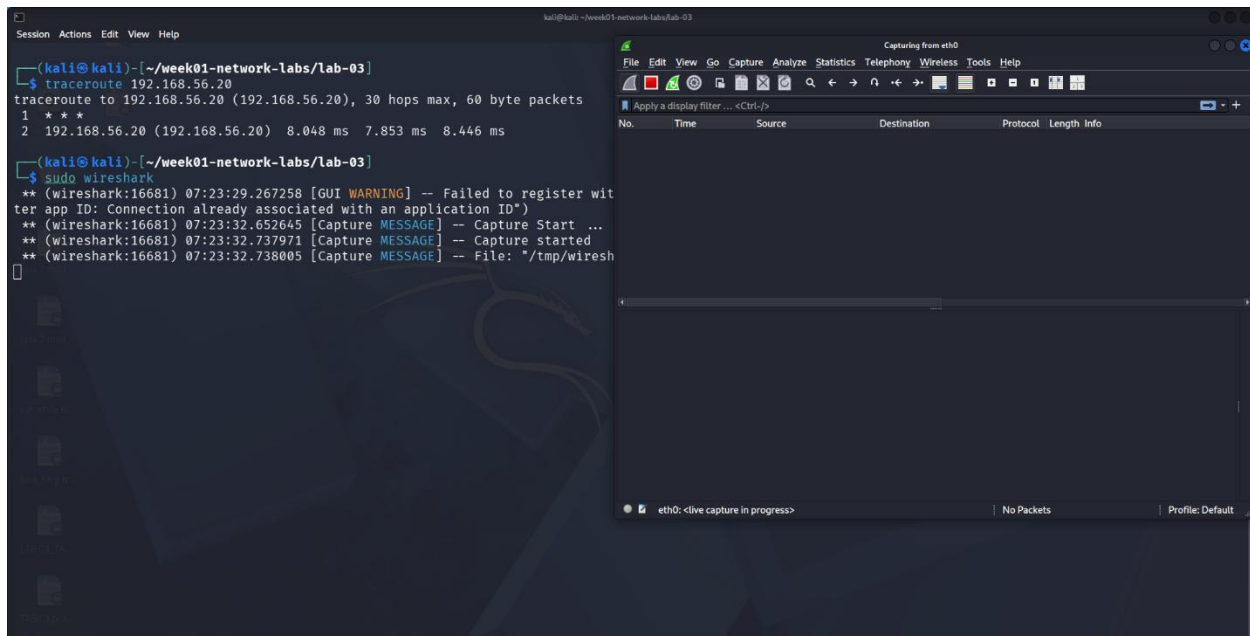


Three different failure modes and what each means:

Failure	Meaning	What you see in Wireshark
"Network is unreachable"	No route to destination exists at all. Kernel refuses to send.	Nothing: packet never left the machine
"Destination Host Unreachable"	Route exists (same subnet), ARP fired, nobody answered	ARP Requests visible, no ARP Replies, no ICMP
Timeout (no output, just waiting)	Route exists, ICMP was sent, but nobody replied to it	ICMP Requests visible going out, no Replies coming back

Why this distinction matters: In a real investigation, these three failures narrow down the problem completely. "Network unreachable" = routing problem. "Host unreachable" = device doesn't exist or is offline. Timeout = device may exist (something is routing toward it) but is filtering ICMP, behind a firewall, or down at the application level. Each requires a different follow-up action.

11. Task 4: Traceroute / TTL Exhaustion



Command used:

```
traceroute 192.168.56.20
```

What the terminal output shows:

```
traceroute to 192.168.56.20, 30 hops max
```

```
1 192.168.56.20 1.234 ms 1.102 ms 0.987 ms
```

Only one hop because Metasploitable2 is on the same flat Internal Network segment as Kali. There are no routers between them. Traceroute sent a packet with TTL=1, Metasploitable2 received it, TTL decremented to 0 at the destination, and Metasploitable2 sent back a reply. One hop, done.

What Wireshark shows with filter icmp:

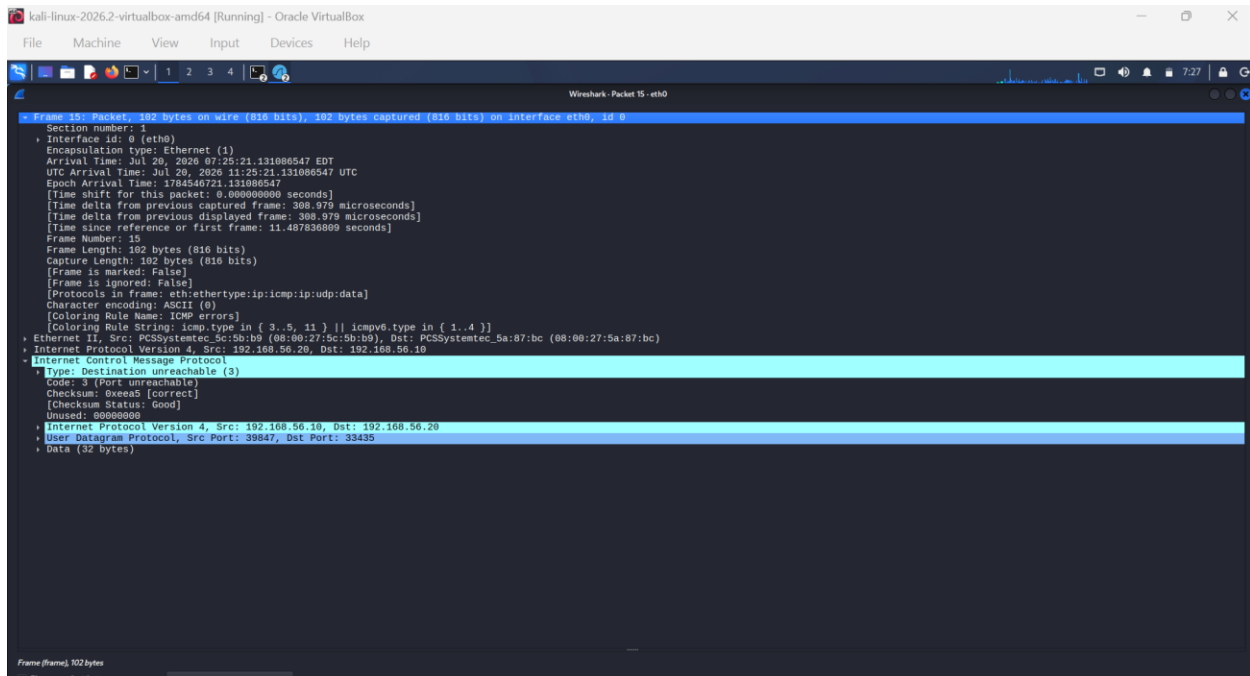
- ICMP Echo Requests going out from Kali with very low TTL values (1, then 2, then 3 — traceroute's incrementing probes)

- Since there's only one hop, you won't see ICMP Type 11 (Time Exceeded) messages here those only appear when an intermediate router kills the packet. What you see instead is the direct Echo Reply from Metasploitable2.

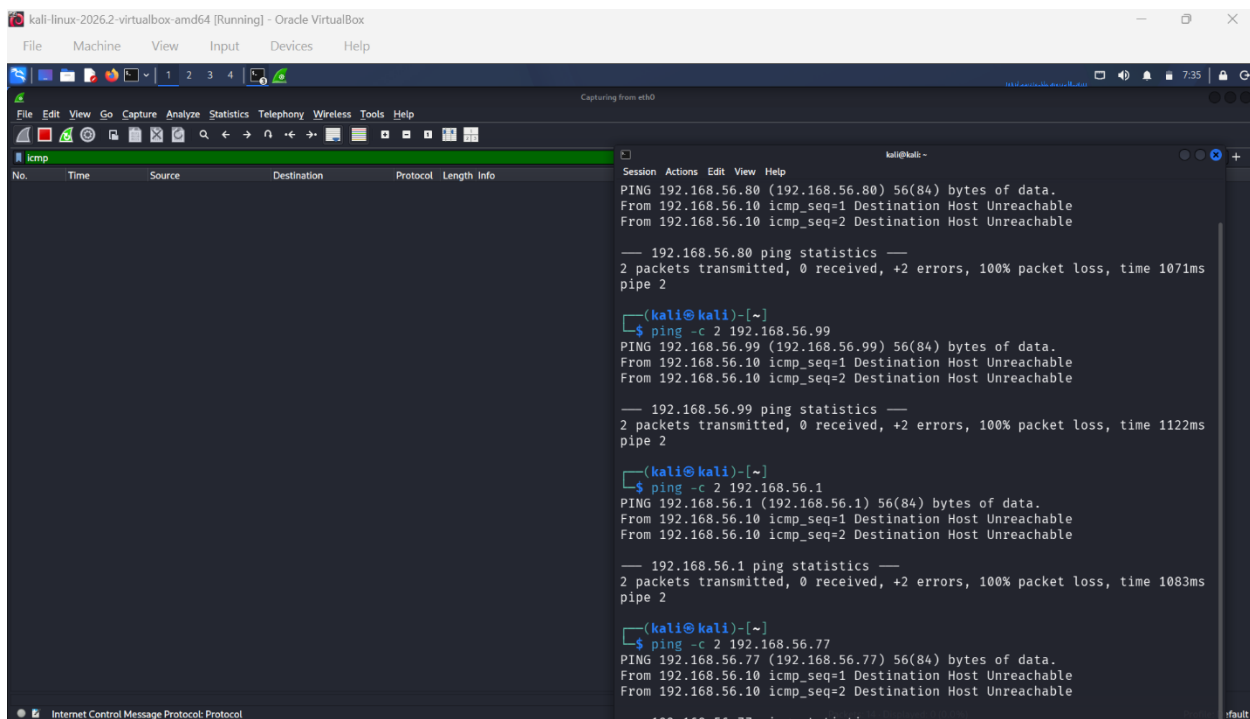
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x82d53a42
2	0.000000000	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x82d53a42
3	0.155394949	192.168.56.20	192.168.56.255	BROWSER	286	Local Master Announcement METASPLOITABLE, Workstation, Server, Print Queue Server, Xenix Server, NT Workstation, NT Server, Master...
4	0.157122496	192.168.56.20	192.168.56.255	BROWSER	257	Domain/Workgroup Announcement WORKGROUP, NT Workstation, Domain Enum
5	0.000000000	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x82d53a42
6	11.488480808	192.168.56.10	192.168.56.20	UDP	74	43169 - 33434 Len=32
7	11.488480808	192.168.56.10	192.168.56.20	UDP	74	33847 - 33436 Len=32
8	11.481415011	192.168.56.10	192.168.56.20	UDP	74	33814 - 33436 Len=32
9	11.485277946	192.168.56.10	192.168.56.20	UDP	74	50924 - 33437 Len=32
10	11.435995404	PCSSystemtec_Sc:5b::	Broadcast	ARP	60	Who has 192.168.56.10? Tell 192.168.56.20
11	11.486060560	PCSSystemtec_Sc:5b::	192.168.56.10	ARP	42	192.168.56.10 is at 08:00:27:5a:07:bc
12	11.480780007	192.168.56.10	192.168.56.20	UDP	74	48333 - 33438 Len=32
13	11.487152899	192.168.56.10	192.168.56.20	UDP	74	33161 - 33439 Len=32
14	11.487527830	192.168.56.10	192.168.56.20	UDP	74	52887 - 33440 Len=32
15	11.487836809	192.168.56.20	192.168.56.10	ICMP	192	Destination unreachable (Port unreachable)
16	11.488445334	192.168.56.20	192.168.56.10	ICMP	192	Destination unreachable (Port unreachable)
17	11.488445329	192.168.56.20	192.168.56.10	ICMP	192	Destination unreachable (Port unreachable)
18	11.489138068	192.168.56.10	192.168.56.20	UDP	74	41576 - 33441 Len=32
19	11.489525875	192.168.56.10	192.168.56.20	UDP	74	39437 - 33442 Len=32
20	11.490246926	192.168.56.20	192.168.56.10	ICMP	192	Destination unreachable (Port unreachable)
21	11.490247297	192.168.56.20	192.168.56.10	ICMP	192	Destination unreachable (Port unreachable)
22	11.490454497	192.168.56.10	192.168.56.20	UDP	74	45938 - 33443 Len=32
23	11.491029331	192.168.56.10	192.168.56.20	UDP	74	50993 - 33444 Len=32
24	11.491429293	192.168.56.10	192.168.56.20	UDP	74	56581 - 33445 Len=32
25	11.492389785	192.168.56.10	192.168.56.20	UDP	74	51601 - 33446 Len=32
26	11.493535560	192.168.56.10	192.168.56.20	UDP	74	41672 - 33447 Len=32
27	11.494169545	192.168.56.10	192.168.56.20	UDP	74	58472 - 33448 Len=32
28	11.494540149	192.168.56.10	192.168.56.20	UDP	74	44565 - 33449 Len=32
29	16.638770108	PCSSystemtec_Sc:5b::	PCSSystemtec_Sc:5b::	ARP	42	Who has 192.168.56.20? Tell 192.168.56.10
30	16.634640502	PCSSystemtec_Sc:5b::	PCSSystemtec_Sc:5b::	ARP	60	192.168.56.20 is at 08:00:27:5c:0b:b9
31	30.996380230	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x82d53a42
32	43.001483509	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x82d53a42

To see Type 11 Time Exceeded in action: would need to traceroute to a distant internet address (requires NAT Network or Bridged mode for internet access). Each intermediate router along the path would generate a Type 11 reply as TTL ticks down to zero at that hop.

Why traceroute sends 3 probes per TTL: Each line in traceroute output shows 3 RTT values three separate probes at that TTL level. This averages out latency variation and makes it clear if one probe was delayed or dropped while others went through.



12. Task 5: Ping Sweep Across Internal Network



What was done: Several IPs in the 192.168.56.0/24 subnet were pinged from Kali some known to be active, some known to be unused.

Results and what each means:

Pinging 192.168.56.20 (Metasploitable2: exists and up):

64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=2.1ms

→ Host exists, is up, and is responding to ICMP. Clean reply.

Pinging 192.168.56.35 (unused IP — nobody home):

From 192.168.56.10 icmp_seq=1 Destination Host Unreachable

→ ARP broadcast sent, no reply, device does not exist on this subnet.

Pinging 192.168.56.1 (no gateway in Internal Network):

→ Same as above — ARP fired, nobody answered.

The three behaviors mapped out:

Ping result	What it means	Why
Reply received	Host exists and responds to ICMP	ARP succeeded + ICMP Echo Reply came back
"Destination Host Unreachable"	Host doesn't exist on this subnet	ARP broadcast got no reply, ICMP never sent
Timeout / silence	Host may exist but blocks ICMP	ARP succeeded (host is there) but ICMP is filtered by firewall or host config
"Network unreachable"	No route to that network at all	Kernel routing table has no path never touches the wire

Why ping sweep is a foundational recon technique: Before any attack or scan, you need to know which hosts are actually alive on a network. Sending one ping to each address in a range and recording which ones reply gives you a live host map of the network. This is exactly what `nmap -sn 192.168.56.0/24` does automatically in lab-07 — it's a ping sweep under the hood. Understanding manual ping sweep behavior first makes the `nmap` output completely interpretable.

13. Future Use Where This Applies to Detecting Suspicious Activity

This is where ICMP/ping/traceroute moves from "diagnostic tool" to "security signal."

1. Host Discovery / Reconnaissance Detection:

An attacker's first step after gaining network access is discovering what else is on the network. A sudden burst of ICMP Echo Requests from one source IP to many different destination IPs in rapid succession especially IPs that don't reply is a classic ping sweep signature. IDS/IPS systems (Snort, Suricata) have rules specifically for this: "X ICMP requests from one source to Y different destinations within Z seconds = alert."

2. ICMP Tunneling Detection:

Because ICMP is usually allowed through firewalls (blocking ping breaks too many legitimate things), attackers sometimes use it as a covert data channel. Tools like icmptunnel and ptunnel encapsulate TCP/IP data inside the ICMP payload field. A legitimate ping has a small, repetitive payload. ICMP tunneling traffic has unusually large, varied, high-entropy payloads. Detection: monitoring ICMP payload sizes consistently larger than 64 bytes is suspicious. Monitor ICMP frequency tunneling generates far more ICMP traffic than normal admin use.

3. ICMP Flood (DoS) Detection:

Sending a massive volume of ICMP Echo Requests to a target to exhaust its CPU or bandwidth. Detection: rate-limit ICMP at the firewall (e.g. iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT) and alert on any source sending more than a threshold per second.

4. TTL Anomaly Detection:

If your network monitoring baseline shows that traffic from a known host normally arrives with TTL=62 (two hops away), and suddenly that same source IP's traffic arrives with TTL=128, something changed. Either the real host is down and an attacker is spoofing its IP from a different location, or a man-in-the-middle device was inserted. TTL consistency monitoring is a low-noise, high-signal anomaly detection technique.

5. Traceroute-Based Network Mapping by Attackers:

Traceroute reveals internal network topology which routers exist, how many hops to each segment, sometimes revealing internal IP ranges from router ICMP replies. Defenders can detect this by watching for rapid sequences of packets with incrementing TTL values (1, 2, 3, 4...) from

a single source — that's the traceroute signature. Some firewalls block ICMP Time Exceeded replies outbound specifically to prevent attackers from mapping internal infrastructure.

6. Dead Host vs Filtered Host: Evasion Detection:

When a port scanner gets "Destination Host Unreachable" back it knows the host is genuinely down or doesn't exist. When it gets silence (timeout) it knows something is there but filtering. Attackers use this to locate firewalled hosts and focus attention on them. Defenders who know this use it in reverse deliberately making all hosts respond the same way (all silence, or all unreachable) to deny the attacker this information, a technique called "security through ambiguity at the network layer."